

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства
Intel – макроопределения»

Выполнила:

Студент 4 курса

Группы АС-66

Рогожин А. С.

Проверила:

Кулик А. Д.

Цель работы: изучение приемов разработки макроопределений использования их в программах

Вариант 20

Постановка задачи. Написать макроопределение, реализующее функцию заданного преподавателем варианта работы.

Задание: переместить заданный символ, если он содержится в строке, в начало строки.

Код программы

```
.386
.model flat, stdcall
option casemap:none

includelib kernel32.lib

ExitProcess PROTO :DWORD
GetStdHandle PROTO :DWORD
ReadConsoleA PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD
WriteConsoleA PROTO :DWORD, :DWORD, :DWORD, :DWORD, :DWORD
SetConsoleCP PROTO :DWORD
SetConsoleOutputCP PROTO :DWORD

STD_INPUT_HANDLE EQU -10
STD_OUTPUT_HANDLE EQU -11

MAX_LEN EQU 100

.data

inputBuffer db MAX_LEN + 1 dup(0)
inputLength dd 0
targetChar db 0
charBuffer db 8 dup(0)
charsRead dd 0

prompt db "Vvedite tekst (do 100 simvolov): ", 0
charPrompt db 13,10,"Vvedite simvol dlya peremesheniya: ", 0
originalMsg db 13,10,"Ishodny tekst: ", 0
resultMsg db 13,10,"Izmenenny tekst: ", 0
newline db 13,10,0
```

.code

PrintString PROC

push eax

push ecx

push edx

push esi

push edi

mov esi, edx

xor ecx, ecx

CountLoop:

cmp byte ptr [esi], 0

je CountDone

inc esi

inc ecx

jmp CountLoop

CountDone:

push STD_OUTPUT_HANDLE

call GetStdHandle

mov edi, eax

sub esp, 4

mov eax, esp

push 0

push eax

push ecx

push edx

push edi

call WriteConsoleA

add esp, 4

pop edi

pop esi

pop edx

```
pop ecx
pop eax
```

```
ret
```

```
PrintString ENDP
```

```
ReadLine PROC
```

```
push ebx
push ecx
push edx
push esi
push edi
```

```
mov esi, edx
mov ebx, eax
```

```
push STD_INPUT_HANDLE
call GetStdHandle
```

```
mov edi, eax
```

```
push 0
push OFFSET charsRead
push ebx
push esi
push edi
call ReadConsoleA
```

```
test eax, eax
jz ReadError
```

```
mov ecx, charsRead
```

```
cmp ecx, 0
je EmptyInput
```

```
RemoveCR:
```

```
cmp ecx, 0
je EmptyInput
```

```
mov eax, ecx
dec eax
```

```
cmp byte ptr [esi + eax], 13
je RemoveLast
```

```
cmp byte ptr [esi + eax], 10
je RemoveLast
```

```
jmp FinishRead
```

RemoveLast:

```
mov byte ptr [esi + eax], 0
dec ecx
jmp RemoveCR
```

FinishRead:

```
mov byte ptr [esi + ecx], 0
mov eax, ecx
jmp ReadExit
```

EmptyInput:

```
mov byte ptr [esi], 0
xor eax, eax
jmp ReadExit
```

ReadError:

```
xor eax, eax
```

ReadExit:

```
pop edi
pop esi
pop edx
pop ecx
pop ebx
```

```
ret
```

ReadLine ENDP

MOVE_CHAR_FRONT MACRO

```
LOCAL SEARCH
LOCAL FOUND
LOCAL SHIFT
LOCAL FINISH
```

```
mov esi, OFFSET inputBuffer
mov ecx, inputLength

xor edi, edi
mov bl, targetChar
```

SEARCH:

```
cmp edi, ecx
jae FINISH

mov al, byte ptr [esi + edi]

cmp al, bl
je FOUND

inc edi
jmp SEARCH
```

FOUND:

```
cmp edi, 0
je FINISH

mov dl, bl
mov eax, edi
```

SHIFT:

```
mov dl, byte ptr [esi + eax - 1]
mov byte ptr [esi + eax], dl
dec eax
jnz SHIFT

mov dl, targetChar
mov byte ptr [esi], dl
```

FINISH:

ENDM

start PROC

```
push 1251
call SetConsoleCP
```

```
push 1251
call SetConsoleOutputCP

mov edx, OFFSET prompt
call PrintString

mov edx, OFFSET inputBuffer
mov eax, MAX_LEN

call ReadLine

mov inputLength, eax

mov edx, OFFSET charPrompt
call PrintString

mov edx, OFFSET charBuffer
mov eax, 7

call ReadLine

cmp eax, 0
je NoChar

mov al, byte ptr [charBuffer]
mov targetChar, al
```

NoChar:

```
mov edx, OFFSET originalMsg
call PrintString

mov edx, OFFSET inputBuffer
call PrintString

MOVE_CHAR_FRONT

mov edx, OFFSET resultMsg
call PrintString

mov edx, OFFSET inputBuffer
call PrintString
```

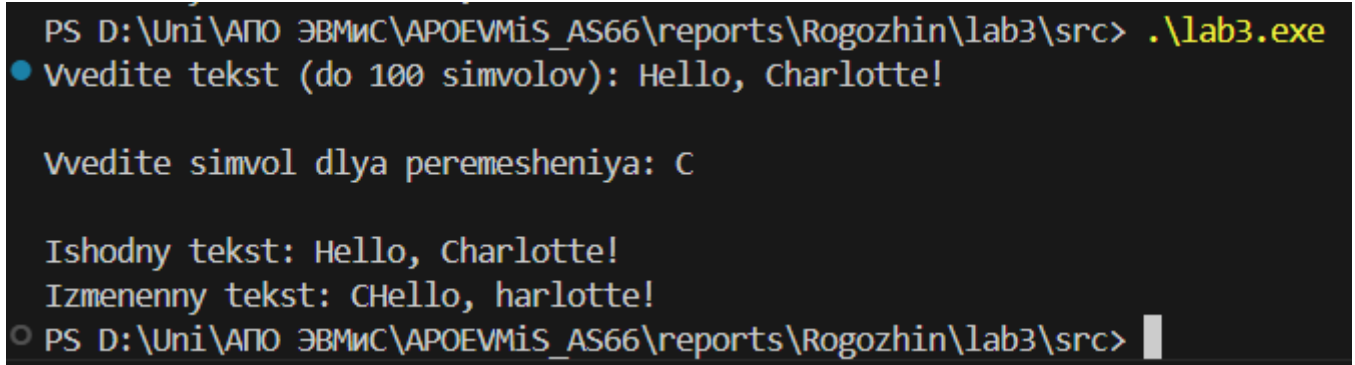
```
mov edx, OFFSET newline
call PrintString

push 0
call ExitProcess
```

```
start ENDP
```

```
END start
```

Результат работы программы:



```
PS D:\Uni\АПО ЭВМиС\АРОЕВМиС_AS66\reports\Rogozhin\lab3\src> .\lab3.exe
● Vvedite tekst (do 100 simvolov): Hello, Charlotte!

Vvedite simvol dlya peremesheniya: C

Ishodny tekst: Hello, Charlotte!
Izmenenny tekst: CHello, harlotte!
○ PS D:\Uni\АПО ЭВМиС\АРОЕВМиС_AS66\reports\Rogozhin\lab3\src> |
```

Вывод: в ходе выполнения лабораторной работы были изучены приемы разработки макроопределений и особенности их интеграции в программы на языке ассемблера MASM (для архитектуры x86). На практике освоен механизм макроподстановок, который позволяет автоматизировать написание повторяющихся фрагментов кода, повысить читаемость исходного текста и оптимизировать структуру программы.